

CUDA PROGRAMMING FOR NVIDIA H100S

内核设计 (**Kernel Design**)

Prateek Shukla · 简体中文版

CUDA kernel 的一般形式

CUDA kernel 有 2 类

- 计算受限 kernel (受算术运算速率限制)
- 内存受限 kernel (受数据搬运速率限制)

分界点就是算术强度 (AI) = FLOPs / Bytes moved

脊点 = Peak FLOPS / Peak BW

~295 FLOP/Byte (FP16 张量运算) ~20 FLOP/Byte (FP32 CUDA core)

- 如果 kernel 的 AI < 脊点 -> 内存受限
- 如果 kernel 的 AI > 脊点 -> 计算受限

计算受限 kernel

计算受限 CUDA kernel 指的是限制因素是计算吞吐量而非数据搬运的 kernel。计算受限 kernel 有以下特征

1. 高算术强度：每加载/存储一个字节就有大量 FLOPs；复用很强：数据保存在寄存器/共享内存中，在被逐出之前被使用很多次，等等。2. 生产者闲置：最显著的特征是生产者/消费者失衡。生产者 warpgroup 闲置在屏障上，等待消费者 warpgroup 追上来3. 占用率“必要但不充分”。许多计算受限 kernel 在中等占用率下就能接近峰值运行，只要它们发射足够多的 WGMMMA 指令。发射计算指令不是问题，寄存器利用率才是。

优化计算受限 **kernel**

以下是一些针对计算受限 kernel 的优化方法

- Warp 专用化
- 常驻 kernel (persistent kernel) + tile 调度
- 带显式同步的共享内存 stage 循环缓冲
- 集群级优化
- 寄存器压力管理
- Megakernel
- 收尾阶段融合 (epilogue fusion)

Warp 专用化

在 Hopper 上，一个 SM 可以容纳许多活跃 warp，但每个周期只有少数 warp 能发射指令

如果同一 warp 内的线程走不同分支，warp 会把这些路径串行化（最坏情况约 32 倍）。但如果 warp 0 负责加载数据，而 warp 1 对这些数据做计算，它们是拥有独立执行上下文的不同 warp，因此你避开了SIMT 惩罚。

通过 warp 专用化，我们有意让同一线程块中的不同 warp 承担不同的工作，典型分工是：

“生产者”warp：搬运/准备数据

“消费者”warp：对这些数据做计算

warp 专用化为何有效

对快速 kernel 而言，warp 专用化几乎是强制的，原因有 3 个 -

1. 资源约束迫使你这样做：若不溢出，单个线程/warp 装不下全部活跃状态（寄存器/谓词等），所以要把工作拆分到多个 warp 上
2. 变延迟操作难以静态调度：内存和其他变延迟操作让编译器/静态调度很难让所有单元保持忙碌。
3. 阻塞式同步否则会拖停发射：如果某个 warp 必须等待屏障等，专用化能让其他 warp 立即运行，这样 SM 就不会浪费发射槽。

资源约束

资源约束——主要是寄存器文件容量与延迟隐藏（latency hiding）需求——是 warp 专用化背后的驱动力。

wgmma kernel 的首要瓶颈是寄存器文件。为了最大化计算吞吐量，线程必须在寄存器中持有输出矩阵的一大块 tile。然而，给每个线程分配 200 多个寄存器会大幅减少能装进一个 SM 的 warp 数量。

WS 把工作解耦成两个角色，允许非对称的资源分配

生产者：它们发射 cp.async 指令，所需寄存器最少

消费者：它们执行 WGMMMA 指令，所需寄存器最多

setmaxnreg

setmaxnreg 指令允许一个 warp 在执行期间动态更改它拥有的寄存器数量。

你可以用较低的寄存器数量启动 kernel。这让 GPU 能在 SM 上容纳许多活跃 warp，最大化内存带宽利用率。

在进入重计算段之前，消费者 warp 执行 setmaxnreg 申请更多寄存器，生产者则执行它来使用更少的寄存器

warp group 数量选择

对生产者，永远恰好分配 1 个 Warp Group。单个线程就能发射一条搬运 GB 级数据的 `cp.async` 指令。

生产者几乎不做计算；它们只管理屏障（`mbarrier`）。你可以用 `set_maxnreg` 把它们限制在约 32 个寄存器。

对消费者，根据寄存器压力在 1、2 或（罕见情况下）3 之间选择。消费者 warp group 的数量由你的累加器 tile 尺寸决定。

预算：约 232-240 个寄存器（为屏障等留出空间）。我们以两种方式使用它

Ping-pong/Basic：每个 WG 独立处理一个完整 tile。EffectiveThreads = 128

协作：两个 WG 分摊同一个 tile。EffectiveThreads = 256

流水线

流水线化就是重叠：你把工作拆成多个阶段，让不同“条目”的不同阶段同时进行，而不是把一个条目从头做到尾再开始下一个。没有流水线化（顺序执行）

加载 -> 等待 -> 计算 -> 等待 -> 加载 -> 等待 -> 计算 -> 等待 ...

每个阶段都阻塞到上一个阶段完成。传输期间硬件闲置。

有流水线化（重叠执行）

Time Load: L0 L1 L2 L3

MMA: M0 M1 M2 M3

在加载 tile N+1 的同时计算 tile N - 硬件保持忙碌

为什么做对很重要

延迟鸿沟：全局内存约 400-800 周期。WGMMA 每次操作约30-60 周期。没有流水线化时，计算等待的时间是运行时间的 20 倍。

没有流水线化时，你的 kernel 是这样：加载一个 tile（计算闲置 400 周期），对它做计算（内存闲置 20 周期），加载下一个 tile（再闲置 400 周期）。你的计算硬件大约只有 5% 的时间在被利用。

有了流水线化，当计算正在啃 tile N 时，内存已经在取 tile N+1。只要流水线足够深，计算就永远不会因为等数据而停顿。你从 5% 走到接近 100% 的计算利用率。这就是全部的游戏。

循环缓冲

要让生产者与消费者重叠，你需要在阶段之间有一个缓冲：一组固定的共享内存“槽位”（stage），由生产者填充、消费者排空。

循环缓冲就是以轮转方式复用这些槽位：

生产者写 stage i ，然后 $i+1$ ，.....回绕回 0。

消费者读 stage i ，然后 $i+1$ ，.....回绕回 0。

这既隐藏了漫长的内存延迟（加载约 400 周期 vs 计算约 20 周期），又让计算持续有输入：这个缓冲正是让你能“在计算 tile N 的同时加载 tile $N+1$ ”的东西。

stage 及其必要性

多 stage 循环缓冲的意义在于让流水线保持忙碌，而不是等待。如果你有 Load -> Compute -> Store 这样的阶段，单个缓冲会强制串行。这浪费时间，因为一次只有一个阶段在工作。多个缓冲通过同时允许多个在途（in flight）数据块来解决这个问题。当 stage 1 在填充缓冲 A 时，stage 2 可以处理缓冲 B，stage 3 可以排空缓冲 C。“循环”指的是一旦一个缓冲走完全程，它就被复用于新输入，在环中回绕。因此你不需要无界的内存，只要足够多的缓冲让所有阶段都有活干。我们需要不止一个缓冲的原因是所有权与重叠。一个阶段不能安全地覆盖另一个阶段仍在读取的数据。分开的缓冲让每个阶段在同一时刻拥有不同的数据。实践中，每个缓冲在若干状态间流转，屏障把所有权从一个阶段转移给下一个。

双屏障握手

循环缓冲中的每个 stage 有两个信号，生产者和消费者因此永远不会竞争：1) FULL 屏障：消费者在读取一个 stage 之前在这里等待。对 TMA 流水线来说，它是事务屏障（transaction barrier）：生产者调用 `mbarrier.arrive`，当那些字节落到共享内存时，TMA 引擎发出完成信号。这样消费者无需轮询就知道该 stage 确实已被填充。2) EMPTY 屏障：生产者在覆盖一个 stage 之前在这里等待。消费者用完该 stage 后向它发信号。生产者等待 EMPTY -> 在 FULL 上声明预期 TX -> 发起 TMA -> TMA 完成 -> FULL 翻转 -> 消费者读取/使用 -> 消费者到达 EMPTY -> stage 可复用

循环缓冲同步中的 **ABA** 歧义

循环缓冲会在多轮之间复用相同的 stage 索引。如果同步只按 `stage_id [0..Stages-1]` 索引，那么观察到“stage 0 已满”的消费者无法判断这个“满”对应的是：

第 1 轮的 stage 0（旧数据），还是

第 2 轮的 stage 0（全部 4 个 stage 回绕之后的新数据）

为了区分同一个 stage 索引的多次使用，流水线跟踪一个 phase bit，每当循环索引从 Stages 1 -> 0 回绕时它就翻转。每个参与者（生产者/消费者）都跟踪一个三元状态：

index : `[0..Stages-1]` 中当前的 stage 槽位；phase : 1 位纪元，回绕时翻转；count : 单调递增的迭代计数器（用于簿记）

CUDA PROGRAMMING FOR NVIDIA H100S

每个 **k tile** 上计数器与 **phase** 的迭代

Prateek Shukla · 简体中文版

CUDA PROGRAMMING FOR NVIDIA H100S

屏障的复用

Prateek Shukla · 简体中文版

生产者流程

每个 warpgroup 只有一个被选出的线程真正接触 mbarrier 和 TMA。每次迭代做三件事：

获取 stage。在 empty 屏障上等待，然后在 full 屏障上调用 expect_tx

发起 TMA 拷贝。当 DMA 引擎把字节写完共享内存后，它会自动向full 屏障发信号。这是硬件级别的生产者-消费者信号传递。

前进。递增流水线状态。回绕时 phase 翻转。

在全部工作结束时，mbarrier.try_wait 会等消费者释放每一个剩余stage，然后 warp 才退出。当还有人还在读你的数据时，你不能退出。

消费者流程

消费者维护两个指向流水线的指针：一个指向当前正在被消费的 stage，另一个指向可以释放回给生产者的 stage。

因为 WGMMMA 是异步 (asynchronous) 的，它不会立即完成。你不能释放它正在读取的缓冲，除非你确定读取已完成。你只能通过 `warpgroup_wait` 才能可证明地确认从某个缓冲的读取已经完成

每次迭代：等数据 (full 屏障) -> 发射 WGMMMA (fence、arrive、gemm、commit) -> 等最旧的 WGMMMA 完成 (`wgmma.wait_group<N>`) -> 释放最旧的缓冲 (empty 屏障) -> 两个指针都前进。

每条 WGMMMA 之前的跨 proxy 栅栏防止编译器把累加器的读/写重排到 WGMMMA 边界之外。

三个阶段

前奏（填充）。消费者连续发射 N 条 WGMMA 而不释放任何缓冲。这会把流水线填满，使得进入稳态时，总有在途的 WGMMA 与 TMA 加载重叠。第一条 WGMMA 把累加器初始化为零。

稳态。吞吐量最优的循环：对每个 k-tile，等数据 -> 发射 WGMMA -> 等最旧的 -> 释放最旧的。TMA 与 WGMMA 完全重叠。生产者始终领先消费者释放指针 Stages 步。

排空。最后一个 k-tile 被消费后，调用 `warpgroup_wait<0>()` 清空所有未完成的 WGMMA，然后释放剩余的 N 个缓冲，让生产者能干净退出。

两类主要流水线

我们构建高性能 kernel 时使用两类主要流水线。Ping-pong 流水线与协作流水线。

协作与 ping-pong 流水线都使用 warp 专用化：把你的线程拆成生产者（负责加载）和消费者（负责计算），给它们一个位于共享内存中的循环缓冲，并让它们同时运行。生产者保持领先，用未来的 tile 填充缓冲槽位，而消费者处理已经填好的槽位。

两种架构的区别在于消费者完成乘法之后、收尾阶段期间发生什么。Ping-pong 把wgmma 操作与收尾阶段重叠，协作模式则不然。

协作流水线 - 架构

384 个线程，拆成 3 个 warp group：

WG0：生产者 - 发起 TMA 加载

WG1：消费者 - 运行 WGMMA，然后执行收尾阶段

WG2：消费者 - 在同一个输出 tile 上做 WGMMA 和收尾阶段。

循环缓冲有固定数量的 stage。每个 stage 有两个屏障：消费者等待的 full 屏障，和生产者等待的 empty 屏障。

生产者以轮转方式填充 stage。消费者以轮转方式读取，偏移量为流水线深度。phase bit 用来区分你正处于缓冲的哪一轮，防止读到陈旧数据。

循环策略

如果你选择的输出 tile 是 128×128 , warpgroup 就分摊这单个 tile 的几何工作负载 :

Consumer 0 计算上半部分 (例如 $M = 0$ 到 63) 。

Consumer 1 计算下半部分 (例如 $M = 64$ 到 127) 。

因为它们在同时处理完全相同的输出 tile , 它们会在完全相同的时刻从共享内存消费完全相同的 A 和 B tile。因为我们使用常驻调度 , 每个 CTA 循环领取一串 tile 每个 warpgroup 走过完全相同的 tile 序列。

如果 tile 就是这么分配的 , 生产者、consumer0、consumer1 都会循环经过 tile T0、T1、T2、T3

CUDA PROGRAMMING FOR NVIDIA H100S

生产者的循环

Prateek Shukla · 简体中文版

CUDA PROGRAMMING FOR NVIDIA H100S

消费者的循环



Prateek Shukla · 简体中文版

协作 - 消费者循环

消费者维护两个指向流水线的指针：读指针和释放指针，二者相差 N （通常为 1）。这个滞后存在的原因是 WGMMMA 是异步的——在读取它的 WGMMMA 真正完成之前，你不能释放缓冲。

我们先在读游标上等待，为该 stage 发射 WGMMMA，并推进读游标。只有当对应的 wgmma 工作完成之后（`wgmma.wait_group`）我们才能安全地释放较旧的 stage 并推进释放游标。读与释放之间的这个滞后正是保护正确性的东西。

K 循环结束后，我们排空未完成的 wgmma 操作，让累加器在寄存器中完全有效。然后收尾阶段开始：我们对这些寄存器累加器运行后处理（如 `scale/bias/activation`），把数据移到共享内存用于某些操作，最后把完成的结果写到全局内存。

协作 **kernel** 不会归约 **WGMMMA** 结果

这是关键洞察。warpgroup 之间的“协作”是共享smem tile，而不是合并累加器。每个 warpgroup 独立地对所有 K tile 执行 wgmma，并累加进自己常驻寄存器的accum。跨 WG 的寄存器通信从不发生——反正这在物理上也不可能，因为 warpgroup 的寄存器是私有的。

每个 WG 写自己互不重叠的 M 区域。收尾阶段利用线程索引（其中编码了 warpgroup），只把该 WG 计算出的那些 M 行TMA-store 到全局输出矩阵的正确区域。

相比 ping-pong 的好处是，一个更大的 tile 可以由两个 WG 共同承担，从而得到更大的有效 MMA tile（256x128 而不是 128x128），同时仍然满足每个 WG 的寄存器预算。

协作 - 缺口

消费者完成一个输出 tile 的全部 k-tile 累加后，它必须做收尾阶段：缩放结果、加 bias、跑激活、存到全局内存。在整个收尾阶段期间，张量核心完全闲置。没有人在使用它们。

C0: [MMA Tile 0] [Epilogue Tile 0] [MMA Tile 1] [Epilogue Tile 1]

^^^^^^^^^^^^^^^^ WGMMA 闲置

对大收尾阶段或小 K 维来说，这段闲置时间占总运行时间的相当大比例。这条流水线把加载与计算完美重叠，但它无法把计算与收尾阶段重叠，因为只有一个消费者身兼两职。

Ping-Pong - 修复方案

增加第二个消费者。当一个消费者在跑收尾阶段时，另一个对下一个 tile 做 MMA。它们交替进行，WGMMA 单元永不闲置。384 个线程，拆成 3 个 warp group：WG0：生产者，WG1：Consumer(C0)，WG2：Consumer(C1) 生产者 warp group 释放自己的寄存器，把更多寄存器文件空间让给两个 MMA warp group。C0: [MMA T0][Epi T0][MMA T2][Epi T2]

C1: [MMA T1][Epi T1][MMA T3][Epi T3]

C1 的 MMA 与 C0 的收尾阶段重叠。C0 的 MMA 与 C1 的收尾阶段重叠。WGMMA 始终忙碌。

Ping-pong 循环

Hopper kernel 通常使用常驻调度。你启动一个较小的 grid（正好装进 SM），每个 CTA 循环领取一串 tile。

假设这个特定 CTA 被分配了 tile 序列：T0、T1、T2、T3、T4、T5。约定如下：

生产者必须处理全部：步长（step size）= 1。

Consumer 0 处理偶数项：步长 = 2，起点 = 0。Consumer 1 处理奇数项：步长 = 2，起点 = 1。

在外层循环内部，你处理某个特定的输出 tile。要为一个输出 tile 做 GEMM，你需要沿 K 维迭代。如果你的 K 维是 4096，而你的 tile 块大小是 $K_TILE=64$ ，那么 $K_TILE_COUNT = 64$ 。

CUDA PROGRAMMING FOR NVIDIA H100S

生产者的循环结构

Prateek Shukla · 简体中文版

CUDA PROGRAMMING FOR NVIDIA H100S

消费者的外层循环

Prateek Shukla · 简体中文版

CUDA PROGRAMMING FOR NVIDIA H100S

内层循环



Prateek Shukla · 简体中文版

Ping-pong 流水线

每个消费者恰好分到 k_tile_count 个 stage——其中 k_tile_count 是 $K / TileK$ 的运行值（对一个输出 tile 要归约多少个 K-tile）。我们通常把 stages 设为固定数字（比如 1、2、3、4），而 k_tile_count 可以大得多（ $K=4096$ ， $TileK=64 \rightarrow k_tile_count=64$ ）

所以在 $Stages=4$ 、 $k_tile_count=64$ 的情况下，实际的 smem 布局是一个由两个消费者复用的循环缓冲：

物理 smem : $[slot0][slot1][slot2][slot3]$ 只有 4 个槽位存在

C0 逻辑 : $pos\ 0,1,2,...,63 \rightarrow$ 映射到槽位 $0,1,2,3,0,1,2,3,...()$

C1 逻辑 : $pos\ 64,65,...,127 \rightarrow$ 映射到槽位 $0,1,2,3,0,1,2,3,...$ (不同 phase)

Ping-pong 流水线

生产者开始用 Output Tile 0 的 K-tile 填充 3 级缓冲。Consumer 0 立即开始它的 MMA。Consumer 1 被 mbarrier 硬阻塞，完全闲置。Consumer 0 啃完 Output Tile 0 的所有 K-tile 后，它在 order 屏障上调用 arrive()

Consumer 0 解除自己收尾阶段阶段的阻塞，开始把 Tile 0 写到全局内存。与此同时，Consumer 1 被解除阻塞。它计算自己的起始逻辑索引（它确切地告诉它，哪个屏障的哪个 phase 对应 Tile 1 的起点），然后开始它的 MMA

Consumer 1 一完成 MMA，就向屏障发信号以解除 Consumer 0 下一个 MMA 的阻塞，而 Consumer 1 转入它的收尾阶段

Ping-pong 流水线与屏障

在有 3 个 stage 的流水线中，硬件不会为每次迭代分配新屏障。它只在共享内存中分配恰好 3 个物理屏障。

Consumer0 总是等待与其输出 tile 0 数据对应的那一代屏障。Consumer1 等待与输出 tile 1 数据对应的那一代。

Consumer0 和 Consumer1 始终相距 `k_tile_count` 个槽位。它们在多次迭代中物理上复用同一个 `full_barrier[i]`，但 phase bit 不同。因为 Consumer 0 和 Consumer 1 处理的是完全不同的输出tile，它们从不同时争抢同一个流水线屏障。

tile 尺寸

输出 tile 决定单个 CTA 计算 C 矩阵的多大一块

目标：让 tile_m 和 tile_n 尽可能大。为什么？因为每次你加载一个 A tile 和一个 B tile，你执行 $2 \times \text{tile_m} \times \text{tile_n} \times \text{tile_k}$ 次数学运算。更大的输出 tile 给你更高的算术强度（从全局内存每加载一个字节换来更多数学运算）。

小/中 tile（例如 128×128 或 64×128 ）：单个消费者 WG（128 线程）的寄存器足够装下累加器。我们使用 Base 或 Ping-Pong

超大 tile（例如 256×128 或 128×256 ）：单个消费者 WG 在物理上装不下用 FP32 表示的 256×128 输出矩阵。这种情况我们一般使用协作模式

内层 **tile** (**tile_K**) 与流水线 **stage**

tile_k 是内层循环每次迭代沿 K 维遍历多“深”。我们把它的尺寸定为既能让张量核心持续有数据、又不撑爆共享内存。要让收尾阶段重叠高效，你的 K 维必须足够大。如果 K 维太小，Consumer 1 会在 Consumer 0 完成收尾阶段之前就完成自己的 MMA。一旦发生这种情况，Consumer 1 又会停顿，你就失去了 ping-pong 设计的吞吐量收益。计算时间必须大于或等于内存写入时间。约束条件：WGMMMA 指令原生消费 K=16 (FP16/BF16) 或 K=16 (TF32)。你需要多个流水线 stage (通常 3 或 4 个) 来隐藏 TMA 加载延迟。

CUDA PROGRAMMING FOR NVIDIA H100S

ping-pong warp 专用化流水线的 phase 跟踪

Prateek Shukla · 简体中文版

warpgroup 内部的角色

生产者 warp group (128 线程 , 4 个 warp) ——每个 warp 有不同的角色 :

- 主循环 DMA warp : 把 A 和 B tile TMA 加载进共享内存
- 收尾阶段 DMA warp : 把 C tile TMA 加载进共享内存 (用于残差相加)
- 调度 warp : 获取下一个 tile 坐标
- 辅助 warp : 可选的额外加载

消费者 warp group 0 (128 线程) ——偶数 tile 的计算 + 收尾阶段

消费者 warp group 1 (128 线程) ——奇数 tile 的计算 + 收尾阶段

ping-pong 的收尾阶段交接

这种交接机制的核心目标是安全、高效的时序编排，确保重计算与收尾阶段刷新在交替的消费者之间无缝发生，不出现数据破坏或流水线停顿。一个专门的生产者 warp 严格管理 `epi_load_pipeline`，发起非阻塞的 TMA 加载，把 Tensor C 之类的元素带进共享内存，用于 $D = A \times B + C$ 这类操作。与此同时，Consumer 0 和 Consumer 1 共享收尾阶段流水线，形成一个共享资源瓶颈，需要精确的时序来避免共享内存中的竞态条件。这些消费者 warp 负责消费 MMA 累加器、应用收尾阶段数学（例如 ReLU 或缩放），并指挥 TMA 单元把 Tensor D 安全地存入全局内存。

带收尾阶段的屏障

ping-pong 设计最神奇的部分是 Consumer 0 的收尾阶段与 Consumer 1 的 MMA 在完全相同的时刻运行。为了安全地做到这一点而不破坏共享内存、不发出冲突的 TMA store，我们使用共享内存中 2x2 的 mbarrier 对象网格。

行 (stage) : 0 = MMA 阶段 , 1 = 收尾阶段 (phase 并不是)

列 (组) : 0 = Consumer 0 , 1 = Consumer 1

每个消费者有一个 group_id (0 或 1)。当消费者调用 arrive() 时，它向当前深度 stage 上另一个消费者的屏障发信号。当它调用 wait() 时，它等待当前深度 stage 上自己的屏障。

深度维度循环：MMA 阶段 -> 收尾阶段 -> MMA 阶段 ->

2x2 网格

每个消费者在每次 tile 迭代中做两个有序操作：先 MMA，后收尾阶段。我们需要强制：1. 同一时刻只有一个消费者做 MMA 2. 同一时刻只有一个消费者做收尾阶段3. 每个消费者对同一个 tile 先做 MMA 再做收尾阶段
barrier_[depth][length] Consumer 0 Consumer 1 depth 0: bar[0][0] bar[0][1] MMA 交接
depth 1: bar[1][0] bar[1][1] 收尾阶段交接每个消费者有一个 group_id。每个操作的协议：

它是如何工作的

你有两个工人（Consumer 0 和 1）和两个区（MMA 和收尾阶段）如果两个工人同时进入同一个区，共享内存会被破坏。如果一个工人等另一个完全做完，张量核心就闲置，你浪费周期时间。2x2 网格通过把阶段解耦来解决这个问题。第 0 行（MMA）：Consumer 0 拿到锁，完成计算，然后向 Consumer 1 发信号。Consumer 1 现在获准开始计算。第 1 行（收尾阶段）：Consumer 0 立即下到收尾阶段行，把它的结果写到全局内存。因为有两条独立的行，Consumer 0 的收尾阶段与 Consumer 1 的 MMA 在完全相同的时刻运行。硬件实现了完美重叠。张量核心永不挨饿

一个 **warpgroup** 单次迭代的完整时间线

1. `ordered_barrier.wait()` 等待 Consumer1 的上一次收尾阶段
2. WGMMMA 主循环 K 循环：从 smem 加载，累加进寄存器
3. `ordered_barrier.arrive()` “我的 MMA 完成了，Consumer1 可以开始它的 MMA”
4. `mma_tail()` `warpgroup_wait<0>`，释放最后的 smem 流水线 stage
5. `ordered_barrier.wait()` 等待 Consumer1 的收尾阶段完成
6. `epilogue.store()` 融合 + R->S 拷贝 + TMA store（细节见后面几页幻灯片）
7. `epilogue.store_tail()` 等待所有 TMA store 落地
8. 推进流水线状态 向前跳 2 步（每个 warp group 各一步）
9. `ordered_barrier.arrive()` “我的收尾阶段完成了，Consumer1 可以开始它的收尾阶段”
10. 取下一个 tile，跳回第 1 步

三条相互独立的流水线

注意计算 warp group 会切换角色：MMA 期间它是主循环流水线的消费者，但收尾阶段期间它变成存储流水线的生产者。同一线程既驱动计算也驱动存储。

CUDA PROGRAMMING FOR NVIDIA H100S

每个消费者 **warp group** 对每个 **tile** 执行的循环：

Prateek Shukla · 简体中文版

收尾阶段 **store** 内部

CTA tile (例如 128x128) 太大, 无法一次性搬到 smem。它被切成若干收尾阶段子块 (例如 64x64 或 128x32)。收尾阶段对这些子块循环:

收尾阶段加载生产者

在这一切发生的同时，收尾阶段 DMA warp（4 个生产者 warp 之一）一直在独立地把 C tile 加载进共享内存：

`load_order_barrier` 确保主循环 DMA warp 在收尾阶段 warp 开始加载 C 之前先加载第一个 A/B tile。此后它们独立运行——收尾阶段 DMA warp 提前填满 C 缓冲，而计算 warp group 在到达收尾阶段阶段时通过 `epi_load_pipeline` 消费它们。

这意味着当计算 warp group 需要 C 数据时，它很可能已经在 smem 里了，从而把 C 的加载与 A*B 的 WGMMA 计算重叠起来。

协作模式的收尾阶段交接

对每个收尾阶段子块，两个消费者 warpgroup 中的每个线程都执行：

步骤 1：加载源矩阵 C（可选）

步骤 2：在寄存器中应用逐元素操作

步骤 3：类型转换

步骤 4：寄存器 -> 共享内存（R2S 拷贝）

步骤 5：共享内存 -> 全局内存（TMA store）

加载 **C** 与存储 **D**

如果源矩阵 **C** 和目标矩阵 **D** 的元素尺寸相同，它们可以共享同一批共享内存缓冲。无需为 **C** 加载和 **D** 存储分配单独的 smem 区域。协议如下：

1. 生产者在 phase **P** 把 **C** 子块加载进 smem 缓冲
2. 消费者从该缓冲读取 **C** (S2R 拷贝)
3. 消费者把 **D** 写进同一个缓冲 (R2S 拷贝)
4. TMA 把 **D** 从该缓冲存储到全局内存
5. 只有在 TMA store 提交之后，我们才把该缓冲释放回给生产者，用于加载下一个 **C** 子块

R2S 拷贝

输出 tile ($\text{CTA_M} \times \text{CTA_N}$) 太大，无法一次性从寄存器搬到共享内存——我们的共享内存不够。所以我们把它切成收尾阶段子块。输出 tile 被划分为更小 tile ($\text{EPI_TILE_M} \times \text{EPI_TILE_N}$) 子块网格，得到一个 2D 网格： M 方向 EPI_M 个子块 $\times$ N 方向 EPI_N 个子块然后用嵌套循环遍历这个网格：`for epi_n in 0..EPI_N:`

```
for epi_m in 0..EPI_M:
```

```
    process subtile (epi_m, epi_n)
```

每次迭代让一个子块走完完整的 寄存器- $\rightarrow$ smem- $\rightarrow$ gmem 路径。两个消费者 warpgroup 都参与每一次迭代——它们的线程在子块的元素之间划分，而不是在不同的子块之间划分。

写回数据

R2S 拷贝之后，每个线程都把它那部分子块写进了共享内存。但 TMA store 需要整个子块就位且可见。所以我们发起一条跨 proxy 栅栏，确保线程的写入可见，然后用 `bar.sync` 在两个消费者 warpgroup 的所有线程之间同步，确保每个线程都完成了自己的 R2S 拷贝和栅栏之后，才允许任何线程继续。

这一切完成后，由第一个 warp 的第一个线程发起向全局内存的 TMA store 操作。我们用 `commit_group` 指令来同步这些操作。发出 `copy + commit_group` 之后，线程并不等待数据到达全局内存。这意味着我们可以把子块 N 的 TMA store 与子块 N+1 的寄存器计算重叠起来。

集成线程块集群

线程块集群是流水线优化中非常重要的一部分。我们一般不会把集群的 DSMEM 特性用于 wgmma 操作，集群的大部分用途与为 wgmma 加载 tile 有关。如果没有集群，计算 C 相邻 tile 的相邻线程块会各自独立地为完全相同的 A tile 发起缓慢的 HBM 读取。

我们用 TMA 多播为共享的 A tile 只对 GMEM 发起一次读取。然后 TMA 单元利用高速的 SM 间网络把该 tile 直接广播到集群中每个线程块的本地 SMEM。这是集群在这条流水线中的唯一用途。

集群内的线程块被保证在同一 GPC 上并发运行。它们可以使用驻留在 DSMEM 中的集群屏障互相发信号。

选择集群尺寸

选择正确的集群尺寸是关键性的调优步骤。因为集群允许线程块直接通信并共享数据，选对形状可以通过减少全局内存流量大幅提升性能

工作负载的形状：又高又瘦（大 M，小 N）：使用偏重 M 的集群

又矮又宽（小 M，大 N）：使用偏重 N 的集群

方形/大型（大 M，大 N）：使用均衡的集群尺寸

一般而言，一个 TPC 内的 2 个线程块通信快得多，因此建议使用 (1, 2) 或 (2, 1) 的集群尺寸来利用这一特性。

集群尺寸为 2

如果 $\text{ClusterSize} = 2$, grid 被划分成一对对 tile。

1x2 集群 ($M=1$, $N=2$) : 两个线程块在 C 的同一行但不同列上计算输出。因此它们总是共享 A tile (A 对应 M 行) , 使用不同的 B tile (B 对应 N 列) 。

2x1 集群 ($M=2$, $N=1$) : 两个线程块在 C 的同一列但不同行上。它们总是共享 B tile , 使用不同的 A tile。

你代码中的常驻循环会改变。步长现在基于你处理多少个集群 tile。但关键在于, 集群内的 CTA 由硬件在物理上保证被共同调度、并发运行。

RS 与 SS

f16/bf16 (2 字节, 等宽, 无 scale) -> 一律 SS: 这些类型拥有广泛的原生 SS GEMM 支持 (包括非 K-major 情况), 因此两个操作数都可以留在共享内存中, 无需预变换步骤。这是最便宜的路径 (寄存器压力更小) 非 2 字节等宽 (tf32/f32/fp8/int8) -> 仅 AkBk (TN) 用 SS, 否则 RS: 对这些类型, wgmma 实际上要求 K-major 喂入。如果输入布局已经是 AkBk, SS 可以直接喂 MMA。如果不是, 你需要操作数交换和/或 B 的转置/重新打包来得到兼容的顺序; 这种灵活性是在 RS (A 来自寄存器 + 转置/交换机制) 中实现的, 因此派发切换到 RS。混合宽度 -> RS: 混合宽度的操作数通常需要在 MMA 之前做转换/反量化。SS 没有前置 MMA 变换阶段。RS 通过对 A 做 smem->register 拷贝提供了这个阶段, 转换可以在这里完成。带 scale / tuple 混合输入路径 -> RS: scale/zero-point 在 wgmma 之前增加了额外的逐元素算术 (例如 $\text{scale} * A$, 可能再 + zero)。这一变换在寄存器中完成, 因此需要 RS

CUDA PROGRAMMING FOR NVIDIA H100S

决策表

Prateek Shukla · 简体中文版

调度

调度是“谁在何时以何顺序做什么”的决策逻辑。它是把工作单元（tile）映射到执行者（线程/warp/CTA/SM）上的具体策略 + 机制。

好的调度能区分“全部硬件忙碌”与“部分 SM 闲置”。它能区分“良好局部性”与“抖动和停顿”。它确保可预测的完成时间，而不是遭受尾延迟悬崖。

调度器把 tile 坐标分发给执行者。标准坐标：批量或分组问题用(M_idx, N_idx, L_idx)。Split-K 坐标：使用 Stream-K 或 Split-K 策略时额外加上 (K_idx, k_tile_count)。

调度为何重要

调度器直接影响三大性能向量：

占用率与利用率：给 SM 喂足够多的独立 tile 以隐藏延迟

负载均衡：避免“长尾”效应——少数 CTA 拿到巨大的 tile，其他 CTA 早早完成并闲置。

局部性与带宽效率：最大化 L2 缓存复用，最小化冗余的全局内存加载，创造更好的多播机会（集群）。

把 tile 尺寸与调度粒度解耦：有了常驻调度，我们有了第三个分解轴：K 维。传统的 tile 并行系统把 $M \times N$ 输出空间划分到各 SM。每个 SM 拥有完整 K 的 tile。这带来不同的权衡，并伴随浪费。

非常驻调度

在标准（非常驻）kernel 中，你启动一个线程块 grid，其中 $\text{Grid Size} = \text{总工作量} / \text{线程块大小}$ 。GPU 硬件调度器把线程块分配给 SM，一个块完成后就退役，硬件再调度新的块。

这套系统有三个问题。第一，你必须多次启动同一个 kernel，这带来 kernel 启动开销.....第二，会出现尾部效应：如果你启动 133 个块而 GPU 有 132 个 SM，那么硬件立即启动 132 个块，等它们完成，然后才只启动剩下的 1 个块

硬件调度器通常是线性的。它们按顺序分配块，Block 0 可能计算左上角 (0,0)，Block 1 可能计算 (0,1)。等硬件轮到下一行 (1,0) 时，所需行的数据已经被从 L2 缓存逐出，因为 grid 在 N 方向上铺得太开。

常驻调度

在常驻调度中，你启动固定数量的线程块（通常等于 SM 数量）。这些块常驻 GPU。它们不在完成一个 tile 后退役，而是进入循环，计算下一个工作 tile 的索引，处理它，持续到所有工作完成。

本课要学三种调度器。它们是：

1. 静态常驻调度器
2. 分组常驻调度器
3. Stream-K 调度器

基线 - 数据并行调度

每个常驻 CTA 以轮转顺序处理 tile :

CTA 0 拿到 tile 0、tile num_SMs、tile 2*num_SMs.....

CTA 1 拿到 tile 1、tile num_SMs+1.....

总工作量 : 150 个 tile , 硬件 : 132 个 SM

总工作量 = 每 SM 1.136 个 tile 单位 , 实际执行 = 2 波 (wave)。

利用率 = $1.136/2 = 56.8\%$

工作内容与非常驻 kernel 完全相同。唯一收益是保证的顺序。波量化 (wave quantization) 依然存在。

消除波量化

132 个 SM (H100)、140 个输出 tile 的情况：

非常驻： $\text{ceil}(140/132) = 2$ 波，第 1 波：132 个 SM 忙碌，第 2 波：8 个 SM 忙碌、124 个闲置 -> 最后一波约 47% 的容量被浪费

剩余 8 个 tile 被均分给全部 132 个 SM。每个 SM 为尾部计算约 0.06 个 tile 的 K 工作量，跨 SM 归约开销被严格限制在最后 8 个 tile 上。

根本洞察在于：波量化不是硬件限制，它是选择 tile 并行分解的后果。常驻调度让你可以选择另一种分解。

静态常驻调度器

这是标准 GEMM 问题默认的高吞吐量调度器。它“静态”是因为工作到线程的映射是数学上预先算好的，而不是通过原子计数器动态认领。

它把输出矩阵看作 tile 网格。它用光栅化 (rasterization , swizzle) 曲线 (通常是 Z 曲线或 U 曲线) 把 tile 分配给常驻线程块。一个常驻块算出它的第一个 tile 索引，计算它，然后向前跳过已启动块的总数 (网格跨步循环 , Grid Stride Loop) 来找下一个 tile。

标准 GEMM、计算受限 kernel 使用这种方法，因为它开销最低 (纯数学，块间无同步) 并最大化缓存命中 (swizzle)。

光栅化

在常驻调度之后，每个常驻 CTA 维护一个线性工作索引。CTA 0 从 tile 0 开始，CTA 1 从 tile 1 开始，依此类推。完成一个 tile 后，每个 CTA 向前跨常驻 CTA 总数步，去领下一个任务

这能工作。它是正确的。而它也会彻底毁掉你的性能。

光栅化是 kernel 中 CTA 被分配到输出 tile 的顺序。这个顺序直接影响 L2 缓存局部性：它确保共享输入数据（A 的行或 B 的列）的相邻 CTA 在时间上接近执行。做错了，你每跨到一个新 tile 就要从 DRAM 重新加载整个矩阵

遍历顺序为何重要：L2 缓存问题

要计算一个输出 tile $C[m, n]$ ，你需要从矩阵 A 的第 m 行加载一条 tile 带，并从矩阵 B 的第 n 列加载一条 tile 带。你沿 K 维流式处理，累加部分结果。

当 A 或 B 的 tile 从 HBM 加载后，它们落在 L2 中，可以被后续 tile 计算复用——但前提是那些后续计算确实需要同样的数据

如果你在一个非常宽的矩阵上从左到右扫描。你握住 A 的第 m 行并遍历 B 的第 0、1、2、.....、 tiles_n-1 列。如果 tiles_n 很大，等你走完这一行、开始第 $m+1$ 行（此时你要复用 B 从 0 开始的那些列）时，那些早期 B 列已经从 L2 被逐出。现在你只好从 HBM 把它们重新加载一遍。

路径策略 - **AlongN** 与 **AlongM**

光栅化根据主轴（major，外层/慢）与次轴（minor，内层/快）使用两种 tile 遍历策略：列主序（column-major）：外层 = N，内层 = M。自上而下走完一个 tile 列，再向右移动。你在固定列 n 上沿 M 向下扫，每个 tile 计算都需要 B 的第 n 列。那一系列在 $L2$ 中保持热度。你在轮换 A 的不同行（这是你付出的代价），但 B 获得最大复用行主序（row-major）：外层 = M ，内层 = N 。从左到右扫过一个 tile 行，再向下移动。你让 A 的第 m 行在 $L2$ 中保持热度，同时轮换 B 的各列。 A 获得最大复用。结论是：你想让哪个矩阵常驻缓存，就沿与它的复用维度垂直的方向遍历。

Swizzle

尽管我们把 A 复用得很好（一行保持不变），我们却在零复用地流式扫过 B——B 的每一列加载一次、用一次，然后被逐出。

当我们来到 M 的下一行重新开始扫描时，需要那些同样的 B 列.....但它们已经不在 L2 里了。我们又回到从 HBM 重新加载 B。我们需要同时在两个维度上都有局部性，让 A 和 B 所需的 tile 都能装进 L2 并被复用多次。

我们使用 swizzle：它修改光栅化路径，让调度器不再沿细线处理 tile，而是处理一个“厚块”的 tile。

我们通过调 swizzle 尺寸来控制厚度：如果 swizzle 尺寸是 1，你得到标准的细光栅。如果尺寸 = 2，swizzle 尺寸是 4，遍历就变成 4 个 tile 厚。

几个要点

你要让次轴（快/内层）成为更长的维度。因为更长的内层循环意味着在迈出“主步”之前有更多迭代，而主步正是昂贵的缓存上下文切换发生之处。

当你在主维度上迈步时，你可能正切到另一个输入矩阵的一条全新条带，这意味着新数据要加载进 L2、旧数据被逐出。

swizzle 尺寸可调。更大的 swizzle = 更强局部性 = 更好的 L2 复用，但收益递减。最佳点取决于 L2 缓存大小、tile 尺寸和矩阵维度。CUTLASS 通常使用 1、2、4 或 8 这类值。

集群与 **swizzle**

在纯 swizzle 中，复用依赖于“相邻 CTA 恰好在时间上接近运行”这一希望。即使逻辑顺序完美，常驻 CTA 也会在不同时刻完成（边缘谓词（edge predication）、不均衡的计算路径、split-K/归约开销）。所以物理执行顺序会偏离逻辑顺序，相邻 tile 可能不再在时间上接近。好的调度器会尽力在这种漂移下仍把 CTA 保持在同一局部性区域内。集群解决了这个问题，因为集群内的 CTA 在空间和时间上一起执行。它们可以同步并协作进行数据搬运。共享的操作数面板可以在集群内的 CTA 之间复用，让复用成为有意为之，而不是碰运气。我们获得更好的 L2 利用率：集群内映射定义了局部共享模式，而集群间光栅化定义了全局复用流向。这在局部和全局两个层级上都最小化了复用距离。

分组常驻调度

这种调度器为 Grouped GEMM 之类的 kernel 设计：你想在单次 kernel 启动中计算多个互不相同的 GEMM。

调度器不把工作负载看作“先 Group 0，再 Group 1……”，而是在概念上把所有组的全部 tile 拼接成一条长线性序列。调度器维护问题的当前组 ID（0、1、2……）、该组开始处的全局线性索引，以及 `total_tiles`：该特定组内的 tile 数量。

各组大小不同。所以当 CTA 的 `linear_idx` 向前推进（推进通常按 `grid-stride`：每个 tile 执行 `linear_idx += grid_size`）时，可能跨越组边界。朴素的标量搜索代价高昂。我们使用 warp 级推测搜索

warp 级推测搜索

如果 `linear_idx` 不在当前组内，warp 以 32 个为一批扫描各组。每个 lane 加载一个组的形状并计算其（集群对齐后的）tile 数量。借助 warp 内建函数（`__ballot_sync`、`__ffs`、`__shfl_sync`），它挑出范围包含 `linear_idx` 的那个 lane，并广播该 `GroupInfo`。如果没有命中，就向前跳 32 个组再来一遍。

找到所属组后，计算局部偏移 $k = \text{linear_idx} - \text{start_linear_idx}$ ，对 k 做 `swizzle -> (cluster_major, cluster_minor)`，按光栅顺序（`AlongM/AlongN`）转换成 (M, N) ，再加上集群内 CTA 偏移。

这次扫描通常很便宜，因为常驻把它摊薄了——大多只在组边界处触发。

慢路径

warp 的各 lane 在不同的组上做推测。lane 0 检查组 G，lane 1 检查 G+1，.....lane 31 检查 G+31 每个 lane 计算其候选组的 tile 数量 (total_tiles)，包括为集群/swizzle 对齐所做的填充。warp 包含式前缀和计算这 32 个候选的累计 tile 数——shfl_sync 循环

每个 lane 推导出自己候选的起始偏移 (start_linear_idx)，并用 ballot_sync 检查 linear_idx < start + total_tiles 是否成立

如果有一个或多个 lane 命中，就选出第一个命中的 lane (__ffs)，把胜者的 group_idx/start/total_tiles 广播给整个 warp (__shfl_sync) 如果全部未命中，就前进 32 个组再来一遍。起始偏移由 lane 31 的累计和更新

数据并行调度的问题

“尾部效应 (tail effect)”：标准调度器把完整输出 tile 分给线程块。如果 tile 总数不是可用 SM 数的整数倍，最后的一“波”工作只被部分填满。在这段尾部波期间，活跃的 SM 处理它们的最后 tile，其余 SM 完全闲置，等 kernel 结束。

这种低效是软件分解问题，不是硬件限制

我们不再把问题看作输出 tile 的 2D 网格，而是引入Stream-K 调度：把整个矩阵乘法视为一条连续的 1D 数学迭代磁带

Stream-K 调度

标准调度器调度输出 tile (M, N)。工作单元是“计算一个完整输出 tile”。Stream-K 调度器调度的是数学迭代，工作单元是特定数量的 MMA 操作。

Stream-K 把第 133 个 tile 拆成 132 份小片，消除了第 133 个 tile 的闲置时间，让所有人在同一时刻完成。调度器算出整个问题所需的总数学运算量，并把这份工作严格均匀地分给所有可用的处理单元。

它按可用处理单元数把这份总工作量严格均分。一个线程块可能计算一个完整 tile，可能接手一个部分完成的 tile，也可能在分到的预算用尽时在 tile 中途停下

混合实现：瞄准尾部

沿 K 维拆分 tile 引入跨块通信开销。把 Stream-K 应用到整个问题往往适得其反。

最优实现是混合方案。调度器让前面的波保持纯数据并行，以获得最大吞吐量。

50% 启发式：Stream-K 拆分只应用于最后的“尾部”波来均衡负载。如果尾部波已经大部分填满（例如 >50%），调度器就退回标准数据并行执行，避免不必要的归约开销。

Stream-K 调度如何工作

调度器计算剩余 tile 中“数学步”的总数。

它把这份总工作量按可用 SM/集群数严格均分

每个集群被分配这条 1D 磁带上的一段特定区间。这产生 3 类工作。大多数时候，集群分到的区间覆盖完整的 (M,N) tile。

一个集群可能接手前一个集群开了头但没做完的 tile。

一个集群可能在 tile 中途用尽分到的预算。它算完该 tile 的前半段 K 循环，保存部分结果，然后停下。

“Fixup”：点对点归约

如果集群 A 算了 Tile X 的前 50%，集群 B 算了剩余的 50%，它们在写全局内存之前需要把结果加在一起。这称为Fixup（修复合并）。系统为这些被拆分的 tile 分配了一个全局内存缓冲区（scratchpad）。

前半段完成它的计算。它把自己的部分累加值写进Workspace 并设置一个标志（Barrier）。后半段完成它的计算。它检查Workspace。如果集群 B 看到集群 A 已完成，集群 B 就加载 A 的部分结果，加到自己的寄存器上，然后把最终的和写到目标矩阵。

反向 **tile** 迭代

当两个线程块共享一个 tile 时，计算 K 维“末端”的块必须等计算“起点”的块完成工作后，才能写最终输出

为了把这段空闲等待时间最小化，Stream-K 工作者按K 的反序遍历分到的输出 tile

通过反向工作，“末端”块把该 tile 中它负责的共享部分安排到执行序列的更晚处。等它准备好合并结果时，“起点”块早已完成，屏障等待时间大幅缩短。

保持 L2 缓存局部性

标准调度通过在几何上聚集工作来保持 L2 缓存局部性。Stream-K 的 1D 磁带方式天然破坏这种空间局部性，可能造成缓存抖动。

为解决这个问题，Stream-K 工作者在逻辑上被分组。调度器把同一组内的单元分配去处理不同输出 tile 上相互重叠的 K 区间。因为它们在同时遍历完全相同的 K 维切片，它们的输入读取在 L2 缓存中完美重叠。

内存受限 kernel

内存受限 kernel 有 3 大类：

- 带宽受限 (bandwidth-bound)：DRAM/L2 吞吐量接近峰值。
- 延迟受限 (latency-bound)：长的记分板 (scoreboard) / 内存等待占主导，

带宽未饱和

- 局部性受限 (locality-bound)：L2 命中差、缓存/TLB 抖动、重间接寻址的访问。

bias、激活、缩放、残差相加、amax 跟踪——在朴素实现里，这些全会是独立的内存受限 kernel。而在好的 kernel 里，它们被折叠进 GEMM 的收尾阶段。你永远不会单独写一个 bias+GELU kernel；你只是在收尾阶段里组合它们。一次全局内存往返，而不是三次。

三大类

带宽受限 kernel：当你已经在从 HBM（有时是 L2）逼近最大持续字节数/秒时，这就是瓶颈；运行时间由你搬运的总字节数和搬运效率主导

延迟受限 kernel：性能主要受一个依赖要花多久才能解决（内存访问、原子操作、同步、长指令链）限制，而不是峰值 FLOPs 或峰值内存带宽。

局部性受限 kernel：你搬了很多字节，但很大一部分因复用差而浪费：L2 命中率低、随机/间接访问、缓存/TLB 抖动等。

收尾阶段

收尾阶段是 GEMM kernel 的最终处理阶段。它发生在 wmma 操作在寄存器中完成之后、数据写回全局内存之前。它把原始累加结果变换成最终输出格式，但我们也可以用它来做其他内存受限操作。目标是在不溢出寄存器的前提下，一边执行必要的数学运算（缩放、bias、激活），一边隐藏延迟。收尾阶段在逻辑上组织成一个对收尾阶段 tile 迭代的嵌套循环。这些 tile 是更大 CTA 输出 tile 的细分。由于矩阵累加结果在 GEMM 主循环后驻留在寄存器中，这层外壳管理数据从寄存器流出、经过共享内存、最终经 TMA 到达全局内存的流程，并管理融合操作应用的粒度。

收尾阶段操作

1. $\alpha * \text{Acc}$ (仅缩放累加器)、 $\alpha * \text{Acc} + \beta * C$ (线性组合)
2. Bias 相加：按行 bias、按列 bias
3. 叠加在线性组合/bias 之上的激活：ReLU GELU SiLU
4. 残差式相加路径 (以残差形式加 source)
5. TopK + Softmax 融合 (按列 softmax 变体)
6. Aux 张量操作：aux load (额外输入张量)、aux store (额外输出张量)
7. 融合进收尾阶段的归约：行归约、列归约、标量归约
8. Absmax/amax 跟踪 (FP8 路径常见)
9. 带缩放的融合：A/B/C/D 缩放的 scale factor、按行/按列的 alpha-beta 缩放变体
10. 块 scale factor 生成变体 (用于块缩放工作流)

收尾阶段中的操作顺序

1. 为该 tile 加载全部所需输入：Acc、可选的 C、bias、scale、可选的 AuxIn。2. 先构建基础表达式： $Z = \alpha * \text{Acc} + \beta * C$ （若无 C 则只有 $\alpha * \text{Acc}$ ）。3. 接着加仿射项：行/列 bias、残差/额外线性项。4. 对 Z 施加非线性：ReLU、GELU、SiLU 等。5. 施加输出缩放/量化（需要时收集 amax）。6. 物化输出：

- 主输出 D
- 可选的 AuxOut（激活前或激活后，按你的需要）
- 可选的行/列/标量归约

7. 最终类型转换 + 存储。

模板

1. 常见前向融合：

$Z = \alpha * \text{Acc} + \beta * C \rightarrow Z += \text{bias} \rightarrow Y = \text{activation}(Z) \rightarrow D = \text{cast/scale}(Y)$ (可选 AuxOut)

2. 训练风格 (保存 aux)：

$Z = \alpha * \text{Acc} + \beta * C + \text{bias} \rightarrow \text{AuxOut} = Z \rightarrow Y = \text{activation}(Z) \rightarrow D = Y$ (可选归约)

3. 反向风格：

$dY = \alpha * \text{Acc} + \beta * C \rightarrow dX = d\text{Activation}(dY, \text{AuxIn}) \rightarrow dBias = \text{reduction}(dX) \rightarrow D = dX$

4. Softmax/TopK 路径 (特例)：

$S = \alpha * \text{Acc} + \beta * C \rightarrow \text{rowmax/rowsum/exp 归一化} \rightarrow \text{可选 TopK} \rightarrow D$ (通常与标准 ReLU/GELU 链分开)

编排器：存储阶段

这一阶段在张量核心的 WGMMMA 计算完成、kernel 从计算转入输出处理时开始。从物理上看，收尾阶段拥有累加器寄存器与最终全局内存写入之间的数据通路。

硬件此时从矩阵乘法发射槽转向收尾阶段指令：读取累加器寄存器、应用输出变换、准备可存储的值。这是输出精度与布局策略第一次被强制执行的时刻。

在 warpgroup 层面发生一次从“WGMMMA 生产者”行为到“收尾阶段消费者”行为的交接，通常由内部排序和屏障保护，确保没有 lane 读到陈旧的累加器状态。一旦完成排序，每个 lane 都可以安全地处理自己寄存器中的元素。

收尾阶段融合生命周期

SM90 的收尾阶段不是单个函数调用。它是一条六阶段流水线，围绕一个原则设计：把会变的与不会变的分开

`begin()` - 一次性的全局初始化

`begin_loop()` - 每个 tile 的设置

`previsit()` - 取操作数

`visit()` - 算术计算（EVT 遍历）

`reduce()`、`postreduce()`、`tma_store()` - 计算之后与存储准备

`end_loop()`、`end()` - 收尾与清理

begin() 与 previsit() - 预置输入

这一阶段预置非累加器操作数（bias、scale、可能还有残差数据）。物理目标是在使用前一刻，把辅助数据从全局内存移进片上存储，再移进寄存器。辅助向量通过 TMA 辅助搬运取入共享内存缓冲。共享内存充当复用的汇合点。随后各 lane 只把自己需要的标量/向量从共享内存加载进寄存器。这让接下来的融合计算保持在寄存器中，避免冗余的全局读取。一个同步步骤确认辅助缓冲已被完全填充，然后任何lane 才消费它们。过了这个屏障，收尾阶段就可以在数据可预测可用的情况下执行融合操作。

算术计算：**visit()**

这是核心的融合计算阶段：缩放、bias 叠加、激活、钳制都直接作用在累加器值上。

每个 lane 读取累加器寄存器，与预置的辅助寄存器结合，逐元素产生变换后的输出。数据通路完全保持在寄存器文件和 ALU/张量执行资源之内。

当前子块的寄存器结果定稿后，lane 交接给存储准备路径，而不是把中间结果写进共享内存。这为真正的提交路径保留了带宽。

主要风险是寄存器过度使用；如果融合深度超出寄存器预算，本地内存溢出（spill）会抹掉融合带来的收益。

典型 **SM90** 融合模式：

碎片化通过显式的 fragment->逻辑坐标映射变得可处理，然后是一条分级的归约流水线
(register -> shuffle -> smem -> gmem/atomic)，视归约范围而定。

计算之后：**reduce()**

FP8 输出引入一个独特需求：你需要计算值的绝对最大值（amax）来设定缩放因子。每个线程在自己的 fragment 上计算本地 amax；各线程汇入共享内存归约缓冲；由单个线程算出最终的 tile 级 amax；该 amax 被写到一个全局缩放因子指针 warp 内归约使用 warp shuffle，让 lane 通过 warp 网络直接交换寄存器值，不产生共享内存流量。这是 warp 范围集合操作中延迟最低的路径。当归约范围超过一个 warp 时，部分和先在共享内存中暂存，再由指定的 lane 或 warp 合并。结果随后被广播回去，让所有生产者能应用一致的缩放或元数据。

存储准备：postreduce()

寄存器到共享内存（R->S）线程用 stmatrix 指令把算好的 fragment 写进共享内存缓冲。这不只是搬运，更是一次变换：Swizzle：数据以特定布局（如 SmemLayoutD）存储，确保TMA 看到连续、对齐的块，而不受逻辑矩阵朝向的影响。精度转换：这是把高精度浮点累加器转成 fp16 存储格式的最后机会。交接边界共享内存充当确定的交接点。数据一旦就位，CUDA core 就从流水线中解放出来，TMA 硬件独立管理到全局内存的异步搬运。

TMA store

消费者 warp 把工作交接给 TMA 引擎。

序列如下：

`fence.proxy.async` - 跨 proxy 栅栏，确保 TMA 能看到数据。

`producer_commit` - 向 StorePipeline 发信号表示一个 stage 已就绪

由一个 leader 线程发起 `cp.async.bulk.tensor`

TMA 接管：Smem -> 全局内存，异步进行

此后我们清理每个 tile 的状态，为下一个 tile 迭代推进流水线屏障；如果使用常驻 kernel，就带着下一个 tile 坐标跳回 `begin_loop()`

完整流水线

`begin()` -> 加载 `alpha`、`beta`，解析谓词。只做一次。`begin_loop()` -> 重新绑定到 `tile (m,n,l)`。每个 `tile` 一次。

`previsit()` -> 取操作数。隐藏延迟。

`visit()` -> 寄存器中的 EVT 数学。热路径。`reduce()` -> FP8 `amax` 归约。有条件执行。`postreduce()` -> 暂存结果：寄存器 -> `smem`。

`tma_store()` -> 硬件搬运 `smem` -> `gmem`。异步。

`end_loop()` -> 推进流水线。下一个 `tile`。`end()` -> 最终冲刷与清理。

CUDA PROGRAMMING FOR NVIDIA H100S

代码与插图

Prateek Shukla · 简体中文版